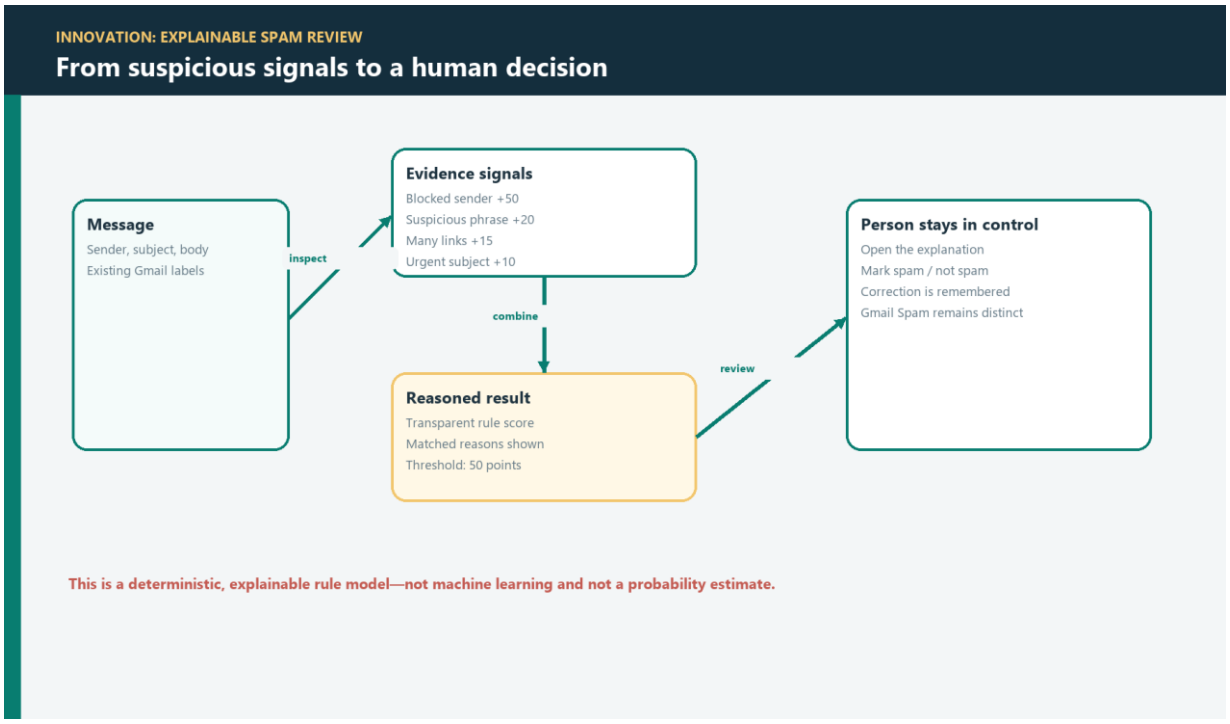
MAILDESK

INTELLIGENT EMAIL MANAGEMENT SYSTEM

A calmer inbox. A clearer decision.



A focused Java desktop client with transparent, user-controlled spam review

INTEGRATED OBJECT-ORIENTED PROGRAMMING + DATABASE MANAGEMENT PROJECT

Submitted by

Member 1: _____ ID: _____

Member 2: _____ ID: _____

Member 3: _____ ID: _____

Programme / Department: _____

Institution: _____

NOVEMBER 2026

BONAFIDE CERTIFICATE

This is to certify that the project report titled “Maildesk — Intelligent Email Management System” is the bonafide work carried out by the students named below as part of the integrated Object-Oriented Programming and Database Management Systems project during the academic year 2026–2027.

The team designed and implemented a Java desktop email workspace with Gmail integration, MySQL persistence, and explainable spam review. The work documented here reflects the current project implementation and its stated verification boundaries.

STUDENT	NAME AND ID
Member 1	_____ / _____
Member 2	_____ / _____
Member 3	_____ / _____

PROJECT GUIDE	COURSE INSTRUCTOR
Signature: _____	Signature: _____
Name: _____	Name: _____
Department: _____	Department: _____

Internal Examiner: _____ External Examiner: _____

ABSTRACT

Maildesk is a Java desktop email-management application built to make everyday mailbox work easier to scan and safer to review. It brings Gmail Inbox, Sent, and Spam into a focused Swing workspace, adds local search and plain-text composition, and stores a bounded mailbox cache and review history in MySQL. Google OAuth is used for account access; a Gmail password is never collected by the application.

The project's distinctive feature is an explainable second opinion for suspicious messages. A deterministic rule classifier adds points for specific evidence such as a blocked sender, suspicious phrases, multiple links, or urgent subject wording. It displays the matched reasons and score beside the message. The score is a rule score, not a probability. A person can mark a message as spam or not spam; that correction is saved for the connected account and takes precedence over the automatic suggestion. Gmail's own Spam folder remains a separate view.

For privacy and predictable demonstrations, connection and refresh import up to six recent messages from each of Inbox, Sent, and Gmail Spam, deduplicate IDs, and cap the local sample at 18 messages. The MySQL schema separates accounts, messages, recipients, labels, spam results, feedback, and blocked senders. Composite account/message keys scope cached records to an account. OAuth tokens and database passwords are kept in local configuration rather than in the database or source repository.

The application uses object-oriented models and service boundaries so the Swing interface, Gmail client, spam rules, and persistence layer can be understood and changed independently. A standalone demo presents fictional messages in memory and makes no network calls or real deliveries. Verification includes unit and integration checks using local fixtures, headless interface checks, and an optional local MySQL check. The current product is a practical student-scale email client, not an enterprise mail server or machine-learning spam platform.

PROJECT AT A GLANCE	CURRENT IMPLEMENTATION
Language and interface	Java 17+, Swing, Maven
Mailbox provider	Gmail API using Google OAuth for Desktop apps
Database	MySQL 8 with JDBC
Review model	Explainable, weighted local rules plus user correction
Safe demonstration	Fictional in-memory sample; no network, database, or delivery

TABLE OF CONTENTS

The sections are arranged to move from the problem and design to the working modules, database, implementation, screenshots, and setup.

CHAPTER	CONTENTS	PAGE
—	Bonafide certificate	2
—	Abstract	3
01	Introduction, problem statement, objectives and scope	5
02	System design, flow, data flow and use cases	6
03	Object model, architecture and shared service contract	8
04	Database model and account-scoped persistence	9
05	Maildesk modules and the explainable review feature	10
06	Application screenshots from the fictional demo	11
07	Implementation approach, safety and verification	14
08	Installation and demonstration guide	15
09	Limitations, future work and conclusion	16
—	References and project links	17

A note on project details

Names, student IDs, department, institution, guide and course-instructor details are left as editable blanks because they were not present in the project source. The example report supplied a different project's institution and student details; those have not been copied into this report.

CHAPTER 1 | INTRODUCTION

1.1 Introduction

Email remains a central channel for study, work, project updates, notifications, and personal communication. Important messages compete with routine announcements and unsafe requests. Maildesk applies a small, clear idea: keep common mail actions together, then give a person a transparent explanation when a message looks suspicious.

Maildesk is a native Java desktop application rather than a hosted website. It connects to a user's Gmail account through Google's OAuth flow, calls the Gmail API, and stores a local application cache in MySQL. The Swing interface provides the workspace; JDBC provides database access.

1.2 Problem statement

A user who wants one focused view of recent mail may need to move repeatedly among mailbox views and search manually for questionable messages. An unexplained automatic filter can also hide a legitimate message or make it difficult to understand why an email was flagged. The project therefore needs to centralize a useful set of mail actions while making suspicious-message suggestions understandable and reversible.

1.3 Objectives

- Connect to Gmail through a browser-based Google authorization flow.
- Present Inbox, Sent, Gmail Spam, and locally suspected messages in one Swing workspace.
- Search the locally cached sender, recipient, subject, and message body.
- Compose and send plain-text email after input validation.
- Persist messages, labels, review results, corrections, and blocked senders in MySQL.
- Explain rule-based spam suggestions and preserve a person's correction.
- Provide a fictional demo mode for safe presentations without Gmail or MySQL.

1.4 Scope

IN SCOPE TODAY	OUTSIDE THE CURRENT BUILD
Gmail Desktop OAuth, recent mail, Inbox/Sent/Spam, local search, plain-text send, spam review and correction.	Attachments, rich-text authoring, scheduled synchronization, hosted multi-user service, or machine-learning classification.
MySQL persistence through JDBC; one connected account context at a time.	Production-scale multi-tenant hosting, enterprise administration, or a complete mail server.
Windows portable package and a standalone fictional demo.	A production Google OAuth verification approval for broad public use.

CHAPTER 2 | SYSTEM DESIGN

2.1 End-to-end system flow

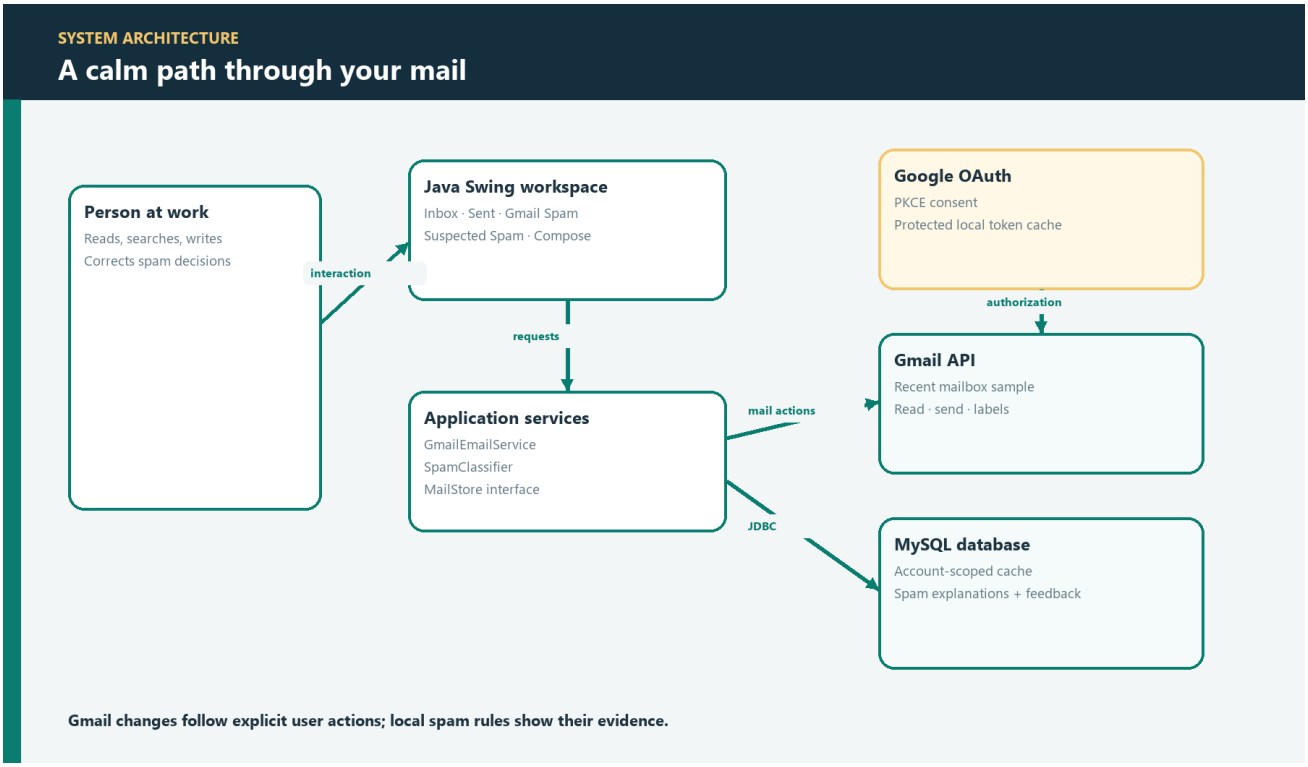


Figure 1. Main application layers and external services

A user action begins in the Swing workspace. The interface calls the EmailService boundary, which routes Gmail actions through the Gmail client and persists mailbox snapshots through MailStore. SpamClassifier evaluates received messages locally. A SwingWorker keeps network and database work away from the interface event thread so the window can remain responsive.

2.2 Data flow

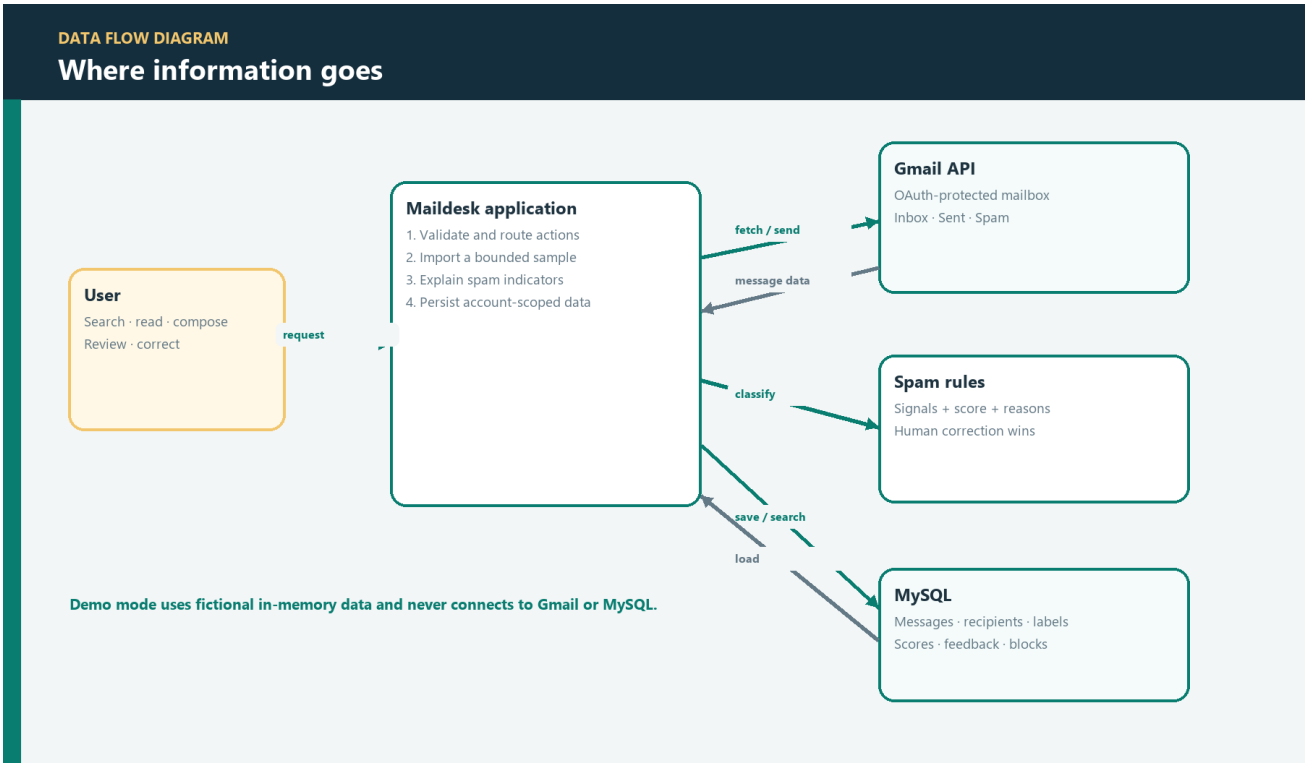


Figure 2. Maildesk data flow; demo mode follows a separate local-only path

2.3 User use cases

ACTOR	USE CASE	WHAT HAPPENS
Mailbox owner	Connect / disconnect	Authorize the Gmail account in Google's browser flow or end the local session.
Mailbox owner	Browse and search	Choose Inbox, Sent, Gmail Spam, or Suspected Spam; search the cached sample.
Mailbox owner	Read a message	Open a message in the reader pane and update its read state through Gmail.
Mailbox owner	Compose and send	Validate one recipient, subject, and plain-text body before sending.
Mailbox owner	Review suspicious mail	Inspect the score and reasons, then mark spam or not spam.
Presenter	Run demo	Explore fictional messages; demo sends stay local and nothing is delivered.

CHAPTER 3 | OBJECT MODEL AND ARCHITECTURE

3.1 Object-oriented design

TYPE / PACKAGE	RESPONSIBILITY
ems.model.EmailMessage	Immutable snapshot of sender, recipients, subject, plain-text body, time, Gmail labels, and account ID.
ems.model.Account / Mailbox / SpamResult	Represent the connected account, mailbox views, and explainable spam result.
ems.service.EmailService	Stable boundary used by the Swing interface for connect, read, search, send, refresh, and correction actions.
ems.service.GmailEmailService	Coordinates Gmail access, local analysis, and the database store.
ems.gmail.*	OAuth, Gmail REST requests, JSON parsing, token storage, and MIME message creation.
ems.storage.MailStore / JdbcMailStore	Persistence contract and JDBC implementation with transactional writes.
ems.ui.*	Swing frames, panels, table model, compose form, and shared visual theme.
com.emailapp.spam.*	Deterministic explainable signal rules and threshold.

3.2 Shared integration contract

The interface contract keeps the view independent of whether data comes from live Gmail or the fictional demo. These method names are the shared vocabulary used by the application and its demo implementation.

```
Account connectGmail() throws AppException;
void disconnect();
void refresh() throws AppException;
List<EmailMessage> getMessages(Mailbox mailbox, String query) throws AppException;
EmailMessage openMessage(String gmailMessageId) throws AppException;
String sendMessage(String recipientEmail, String subject, String body) throws AppException;
SpamResult getSpamResult(String gmailMessageId) throws AppException;
void markAsSpam(String gmailMessageId) throws AppException;
void markAsNotSpam(String gmailMessageId) throws AppException;
```

3.3 Application sequence

1. The user selects Connect Gmail and completes Google authorization in the browser.
2. Maildesk retrieves up to six recent messages from Inbox, Sent, and Gmail Spam; duplicate IDs are removed and the sample is capped at 18.
3. The service analyzes received messages and stores the account-scoped snapshot and result in MySQL.
4. The user reads, searches, composes, or reviews; explicit Gmail actions and saved corrections update the connected account.

3.4 Error handling and responsiveness

Slow service work runs in background workers so the Swing interface stays responsive. Recoverable errors appear as clear status messages. Gmail reads use bounded retry for temporary rate limits; a failed refresh preserves the signed-in session and previous cache.

CHAPTER 4 | DATABASE DESIGN

4.1 Entity–relationship diagram

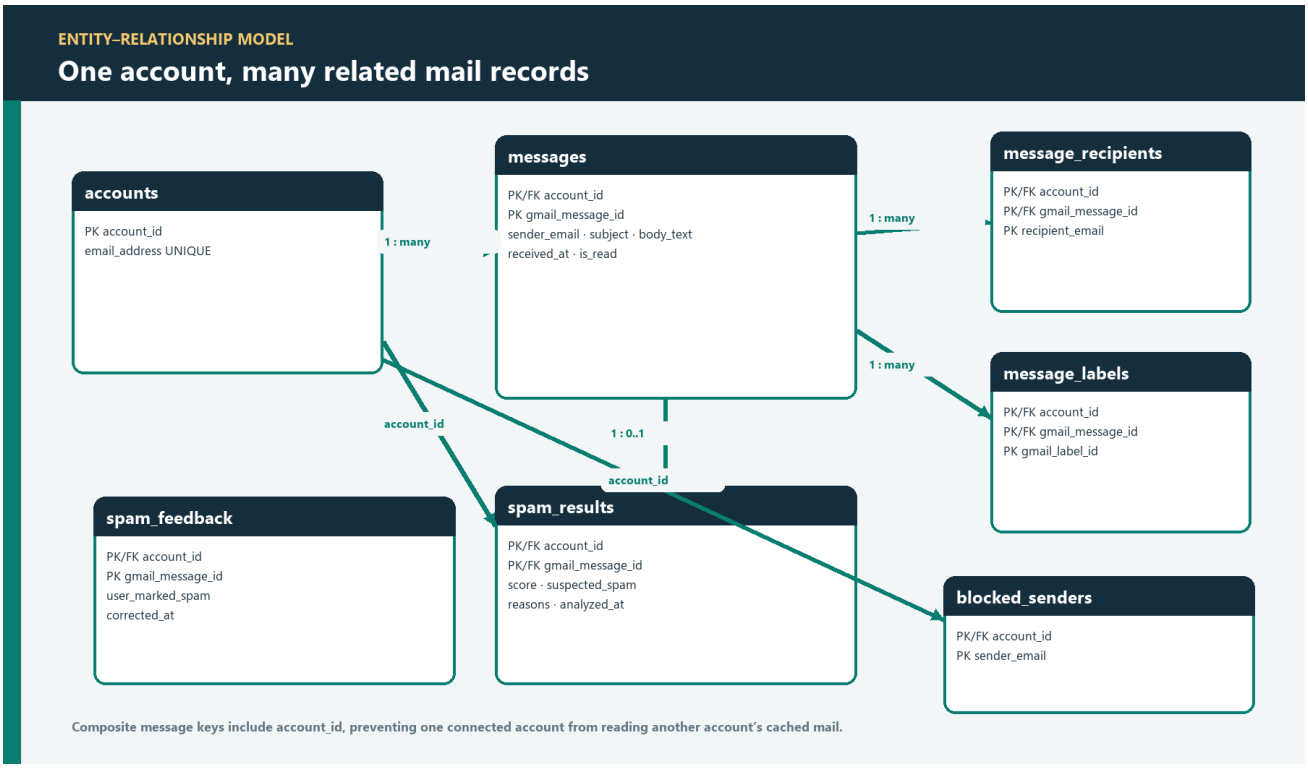


Figure 3. Logical schema based on sql/schema.sql

4.2 Tables and keys

TABLE	PURPOSE	IMPORTANT RELATION
accounts	Connected Gmail account identity.	Unique email_address; parent for account-owned data.
messages	Cached message headers, body, date, and read state.	Composite primary key (account_id, gmail_message_id).
message_recipients	One row for each recipient.	Foreign key to the composite message key.
message_labels	Gmail label IDs attached to each cached message.	Foreign key to the composite message key.
spam_results	Rule score, suspected flag, explanation, and analysis time.	One result per account and message.
spam_feedback	User's spam/not-spam correction and time.	Scoped to account; survives sample replacement.
blocked_senders	Account-specific blocked sender addresses.	Composite key (account_id, sender_email).

Foreign keys connect child rows to accounts and messages. Composite message keys scope each record to an account. JDBC uses prepared statements and transactions for related writes and snapshot replacement; startup initializes tables when the configured database user has permission. OAuth tokens stay in a protected local token store, not in MySQL.

CHAPTER 5 | MODULE DESCRIPTION

5.1 Gmail authorization

The app uses a Google OAuth Desktop client and a system browser. Its authorization flow uses PKCE, a random state value, and a loopback callback. The user signs in directly with Google; Maildesk does not ask for or store the Gmail password. Local tokens are kept outside the project's tracked source files.

5.2 Inbox workspace

The main window combines a navigation rail, search field, message list, and reader pane. Inbox, Sent, Gmail Spam, and Suspected Spam have distinct meanings. Search covers the current cached sample and matches sender, recipients, subject, or body. Refresh updates the bounded local sample; it does not delete messages from Gmail.

5.3 Plain-text composition

The compose form validates a single recipient address, a subject from 1 to 200 characters, and a non-empty body within the configured size limit. A failed send leaves the draft available to retry. Attachments and rich-text formatting are not in the current build.

5.4 Explainable spam review

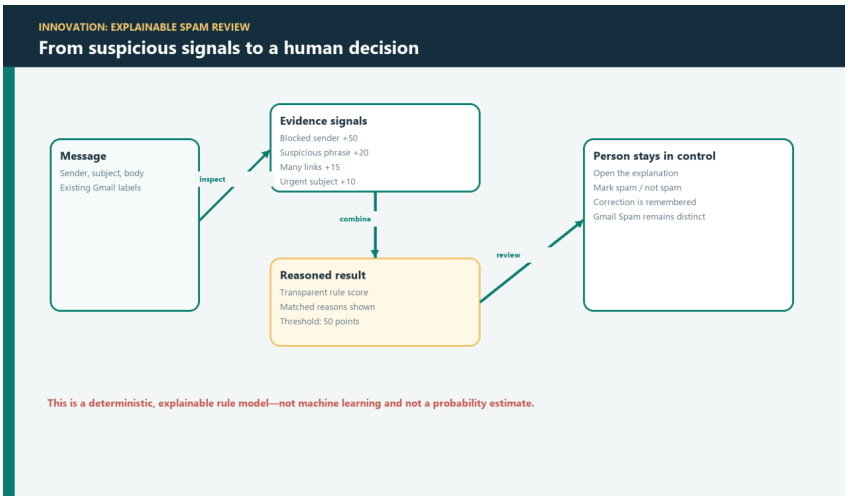


Figure 4. The user sees the signals and makes the final correction

SIGNAL	POINTS	HOW IT IS EXPLAINED
Blocked sender	+50	"Sender is blocked."
Each matched suspicious phrase (up to three)	+20	Shows the phrase that matched.
Five or more links	+15	"Message contains many links."
Urgent subject wording	+10	"Urgent subject wording."
Review threshold	50	At or above threshold is suggested spam; score is not probability.

A saved correction overrides later suggestions for that account. Gmail Spam and Maildesk's local Suspected Spam review remain separate.

CHAPTER 6 | APPLICATION SCREENSHOTS

The screens below are from Maildesk's standalone Swing demonstration with fictional sample messages. The screen showing student@example.test is a demo identity, not a real mailbox.

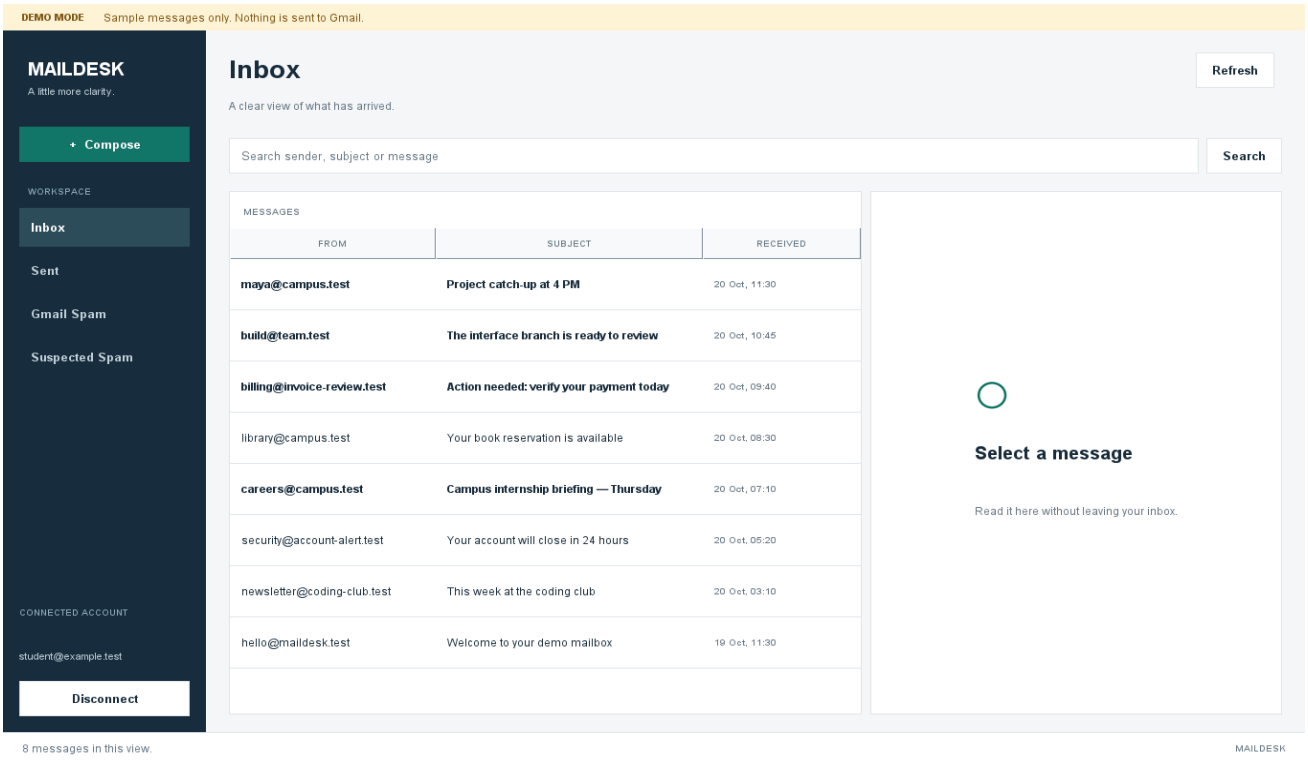


Figure 5. Mailbox workspace: navigation, search, message list, and reader pane

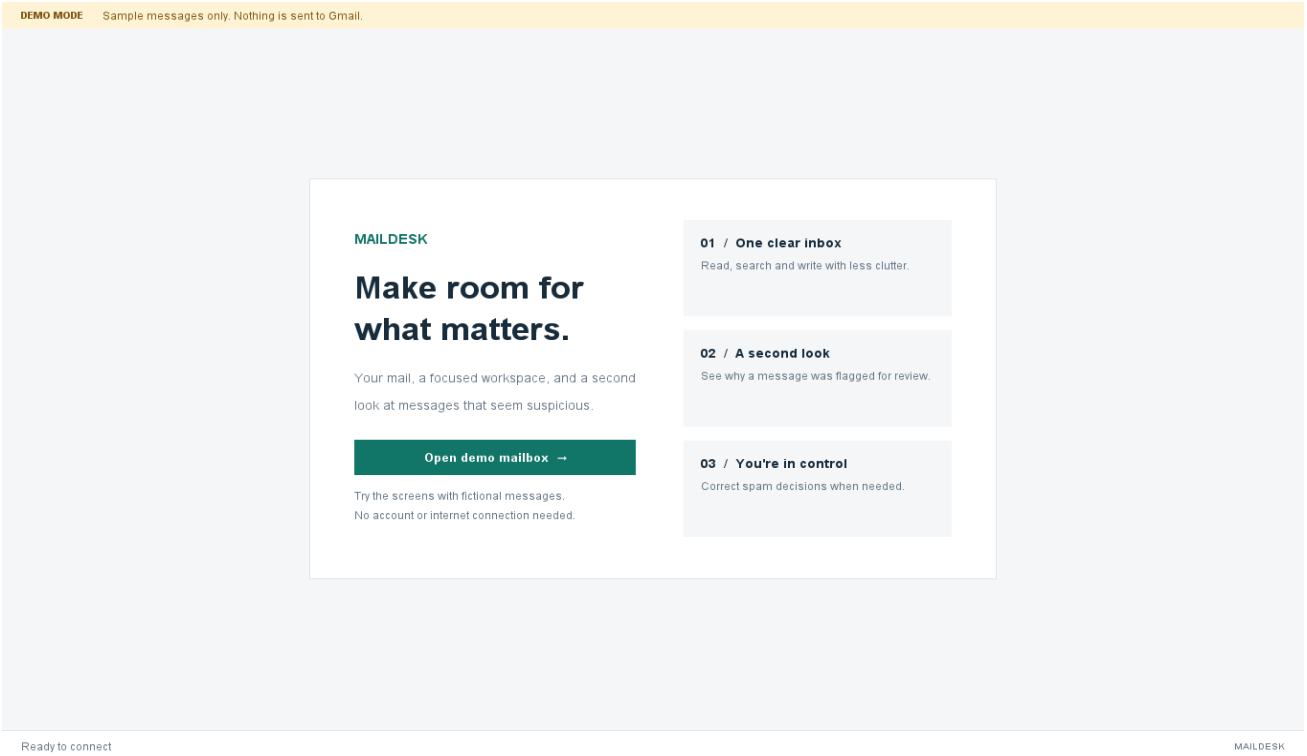


Figure 6. Welcome screen explains the three product ideas and offers a safe demo entry

6.1 Reading a message

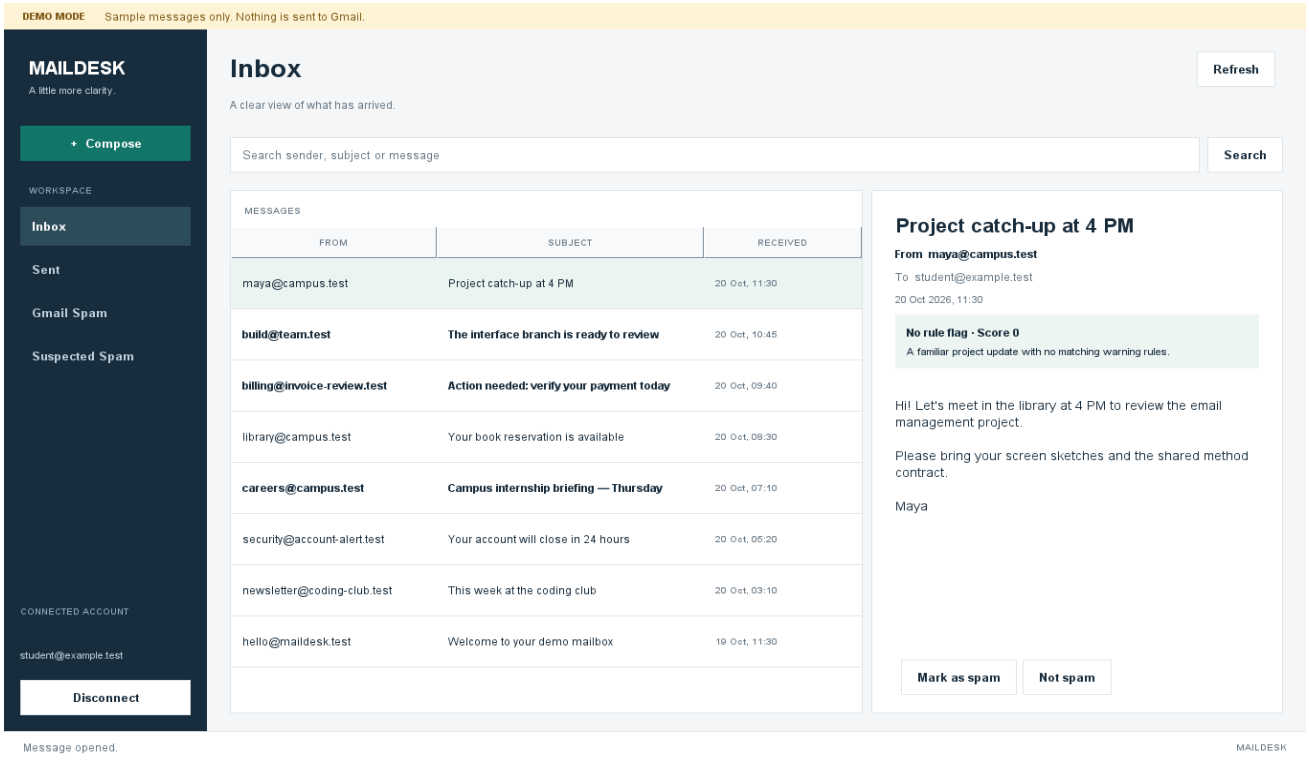


Figure 7. Reader pane displays a message and its rule review beside the inbox

6.2 Suggested spam

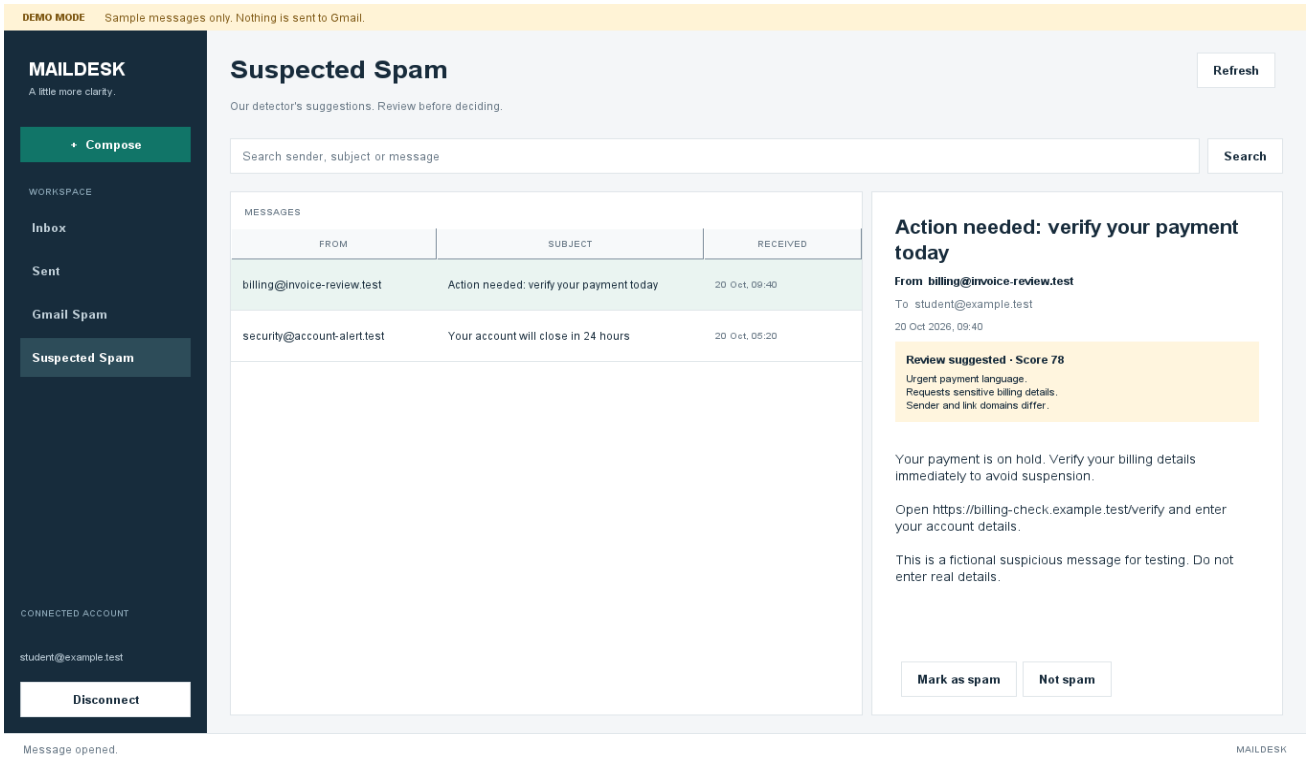


Figure 8. A fictional invoice message shows its score, evidence, and correction controls

6.3 Compose

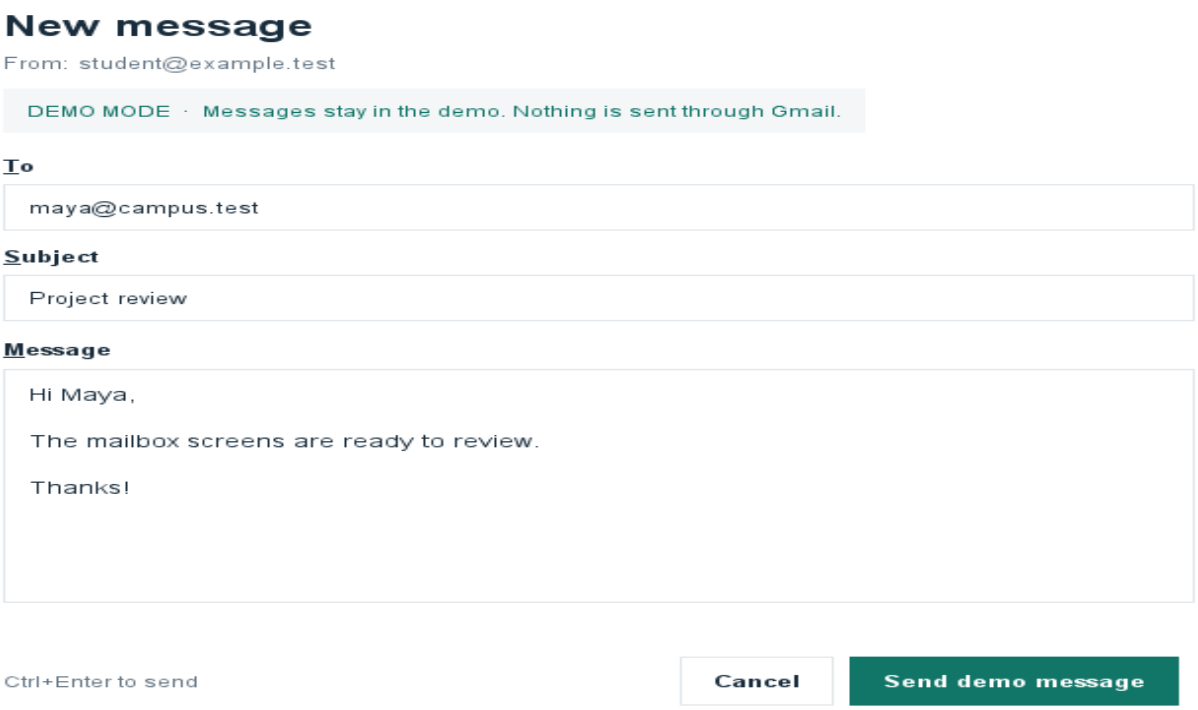


Figure 9. Plain-text compose preview; demo send remains in memory

All demonstration addresses use the reserved .test domain. Real Gmail operations require the configured credentials, Google consent settings, network access, and a running MySQL database.

CHAPTER 7 | IMPLEMENTATION, SAFETY AND VERIFICATION

7.1 Technology stack

LAYER	TECHNOLOGY / RESPONSIBILITY
Desktop interface	Java Swing; layouts and message reading are implemented with reusable panels.
Application boundary	EmailService interface and domain models.
Gmail integration	Google OAuth Desktop flow and Gmail REST API; MIME helper for plain-text sends.
Persistence	MySQL 8 through JDBC; Maven-managed MySQL Connector/J.
Spam review	Local Java rule classifier with visible score reasons and feedback.
Packaging	Maven executable JAR and a Windows app image with bundled Java runtime.

7.2 Security and privacy choices

- Use Google sign-in in the system browser; do not collect the mailbox password.
- Keep OAuth client JSON, local tokens, database passwords, and private email exports out of Git.
- Use account-scoped tables, foreign keys, parameterized JDBC, and transactions for related writes.
- Use a dedicated local MySQL account with only the needed database permissions.
- Import only a bounded recent sample; refresh changes the local cache, not Gmail's mailbox contents.
- Keep local suspected-spam decisions separate from Gmail Spam and explain the matched rules.
- Run the fictional demo for presentation when real account access is unnecessary.

7.3 Verification

The repository includes JUnit checks for OAuth callback handling, HTTP responses, JSON and MIME parsing, Gmail service behavior, spam classification, configuration, and JDBC persistence. Contract and headless Swing checks exercise user-facing actions through the same service boundary. The default suite uses local fixtures and an isolated test database. A separate opt-in check can exercise the configured MySQL server. Automated checks do not send real email.

GitHub Actions builds and verifies on Windows and Linux with JDK 17. A native Windows packaging job also checks the packaged launcher. The live Gmail and local MySQL setup are environment-dependent; the demo mode is the reproducible, offline presentation path.

7.4 Known limits

CURRENT LIMIT	PRACTICAL EFFECT
One connected account at a time	Operations are scoped to the active account.
Recent sample capped at 18 unique messages	A focused cache, not a full Gmail mirror.
Plain-text email only	No attachments or rich formatting.

CHAPTER 8 | INSTALLATION AND DEMONSTRATION

8.1 Fast classroom demo

5. Run the interface demo using the repository script: .\scripts\run-demo.ps1.
6. Choose “Open demo mailbox” to enter the fictional inbox.
7. Open Inbox, Sent, Gmail Spam, and Suspected Spam; inspect a message explanation.
8. Try search and the compose form. The demo labels local sending clearly and no message is delivered.
9. Use the safety banner and the reserved .test addresses to explain that this mode is fictional.

8.2 Configure live Gmail and MySQL

10. Install JDK 17 or newer and MySQL 8; start the MySQL server.
11. Copy config/app.properties.example to config/app.properties and set a dedicated local database user and password there.
12. In Google Cloud, enable Gmail API and create a Desktop OAuth client. Configure the consent screen and add the account as a test user while the application is in testing.
13. Save the Desktop OAuth JSON as config/credentials.json. Never commit it or paste its contents into a report, issue, or chat.
14. Build with .\mvnw.cmd --batch-mode clean package. Start with .\scripts\run-app.ps1 -NoBuild.
15. Click Connect Gmail, complete Google’s browser consent, then refresh and inspect the cached sample.

8.3 Windows package

The project provides a portable Windows x64 ZIP from the GitHub Release. Extract the entire archive and run Maildesk.exe. The package includes its Java runtime; local MySQL and OAuth setup are still required for a real account. Do not distribute the personal config folder or token cache with the package.

8.4 Review database records

After live setup, a reviewer may inspect the project database with a database client. Example read-only queries:

```
USE email_management;
SHOW TABLES;
SELECT COUNT(*) AS cached_messages FROM messages;
SELECT gmail_message_id, score, suspected_spam FROM spam_results;
```

The live MySQL acceptance check is optional and depends on a correctly configured local server. A successful demo-mode run alone does not prove that Gmail OAuth or MySQL credentials are configured.

CHAPTER 9 | FUTURE WORK AND CONCLUSION

9.1 Future work

IDEA	WHY IT MATTERS	STATUS
Attachment preview and safe download	Let people inspect common files while keeping content handling explicit.	Future enhancement
Richer composition and drafts	Support formatted mail and draft recovery.	Future enhancement
User-tunable rule weights	Adapt the explainable scoring to a user's own corrections.	Future enhancement
Privacy-preserving classifier evaluation	Measure false positives on consented test mail before changing rules.	Future enhancement
Scheduled synchronization and notifications	Reduce manual refresh while preserving clear user control.	Future enhancement
Broader Gmail OAuth verification	Prepare a reviewed public distribution path beyond test users.	Deployment work

A useful next milestone is to evaluate the detector with a labeled, consented set of fictional or owner-provided samples. The system should report false positives and keep a person in the decision loop before adopting any model beyond the current transparent rules.

9.2 Conclusion

Maildesk brings together Java desktop development, object-oriented design, Google authorization, Gmail API integration, and relational database work in one coherent application. It offers a focused mailbox with reading, search, and plain-text sending; a MySQL cache organized around account-scoped records; and a spam review view that shows how its suggestions are formed.

Its strongest project contribution is the combination of a bounded mailbox sample, transparent spam explanations, and user correction. This makes the system practical to demonstrate and easier to reason about than a hidden filter. The safe offline demo gives the team a reliable presentation path, while the live client remains configurable for a student's Gmail and MySQL environment.

The implementation is a working academic prototype with clearly stated limits. Attachments, rich text, scheduled sync, machine learning, multi-account administration, and broad public OAuth approval remain future work. The architecture provides a clear route to extend it without obscuring what exists today.

REFERENCES

1. Maildesk source repository, setup guide, architecture notes, and SQL schema. https://github.com/Sanjay-AI-ML/email-management-work

2. Google for Developers. Gmail API documentation. https://developers.google.com/workspace/gmail/api

3. Google Identity. OAuth 2.0 for native and desktop applications. https://developers.google.com/identity/protocols/oauth2/native-app

4. Google for Developers. Gmail API authorization scopes. https://developers.google.com/workspace/gmail/api/auth/scopes

5. Oracle. Java SE documentation. https://docs.oracle.com/en/java/

6. MySQL. MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en/

7. Project implementation files: src/main/java/ems, src/main/java/com/emailapp/spam, sql/schema.sql, docs/SETUP.md, and docs/ARCHITECTURE.md.

APPENDIX A | PROJECT LINKS

Source code and Windows package: https://github.com/Sanjay-AI-ML/email-management-work

Setup guide: docs/SETUP.md · Architecture: docs/ARCHITECTURE.md · SQL: sql/schema.sql